

AMONG BOTS

[ILLUSTRATES.DEV](#)

[AMONG-BOTS-GITHUB](#)



Februar 9. 2026

HFT STUTTGART

[Schellingstraße 24,](#)
[70174 Stuttgart](#)

Prepared By:

- **Akansha Darshi** – Logic Programming, Merging – GitHub
- **Hayder Abed** – ML Toxicity Model – GitHub
- **Maksym Pavlovskyi** – Docker Setup, Hosting – GitHub
- **Muhammed Natheer Abdullah** – AI Toxicity Model - GitHub
- **Roland Fazekas** – Logic Programming, AI Bots - GitHub
- **Ufuk Emre Sen** – UI Programming – GitHub

Datum: Februar 9. 2026

Inhaltsverzeichnis

Zusammenfassung.....	3
Projektziele	3
Leistungsumfang.....	3
Stakeholder	3
Ressourcen	3
Risikomanagement	3
Kommunikation.....	4
Qualitätsmanagement.....	4
Datenschutz & Moderation	4
Application Architecture	4
Modulen	6
Spiel Logik.....	6
Game State (Frontend)	6
Socket Events (Backend)	7
AI Spieler Logik.....	7
Toxicity Filter	8
Mögliche Weiterentwicklungen.....	10



Zusammenfassung

Among Bots ist ein Multiplayer-Social-Deduction-Spiel, in dem echte Spieler:innen in einer Gruppe gegen LLM-Modelle als Gegenspieler antreten. Ziel ist sowohl Unterhaltung als auch das Sammeln von Intuitionen und Erkenntnissen über KI-Täuschung in einer kontrollierten, spielerischen Umgebung.



Projektziele

- **Soziales Multiplayer-Erlebnis:** Entwicklung eines spielbaren, webbasierten Spiels, das gezielt für Gruppen von Freund:innen konzipiert ist, da eine bestehende soziale Dynamik die gewünschte Nutzererfahrung ermöglicht.
- **KI:** Entwicklung von KI-Agenten, die menschliches Verhalten ausreichend realistisch nachahmen, um menschliche Spieler:innen innerhalb einer geschlossenen Lobby täuschen zu können.
- **Orchestrierung:** Entwicklung der Anwendung mit Fokus auf einfache Bereitstellung und Skalierbarkeit mittels Docker, zugänglich über gängige Webbrowser.

Leistungsumfang

In-Scope: WEB: FrontEnd, BackEnd, AI integration, Containerized with Docker, Hosted on DigitalOcean(Droplet: VM), proxy with NGINX, DNS & Real-time Analytics with Cloudflare;

Out-of-Scope: Mobile apps (on IOS/Android), physical games.

Stakeholder

Teammitglieder, Nutzer:innen, akademische Betreuungsperson.

Ressourcen

Teamrollen: Frontend, Backend, grundlegendes DevOps sowie KI/ML.

Tools: TypeScript, React, [Node.js](#), [Express.js](#), Webpack(for UI build), [Socket.io](#), Docker, OpenAI API, GitHub.

Risikomanagement

Identifizierte Risiken:

- **API-Kosten:** Die Nutzung externer KI-APIs (z. B. OpenAI) kann erhebliche Kosten verursachen, insbesondere bei hoher Spieler- oder KI-Aktivität.

- Toxische Inhalte: Spieler- oder KI-generierte Inhalte können toxische oder unangemessene Sprache enthalten.
- KI-Erkennung: Menschliche Spieler können KI-Teilnehmer erkennen, was die Spielimmersion verringert.

Strategien zur Minderung:

- Kostenkontrolle: Einsatz von Filtern und Rückfalllogik, um unnötige API-Aufrufe zu minimieren
- Inhaltsmoderation: Integration fortschrittlicher Toxizitätsfilter und Reinforcement Learning from Human Feedback (RLHF) zur Reduzierung schädlicher Ausgänge.
- KI-Tarnung: Kontextbasiertes Verhalten und Output von KI-Agenten, um menschliche Spieler besser nachzuahmen.



Kommunikation

Group chats und GitHub Pull requests

Tools: Zoom, Teams, GitHub, WhatsApp.

Qualitätsmanagement

Code quality via TypeScript.

Reviews via testing und controlled merges.

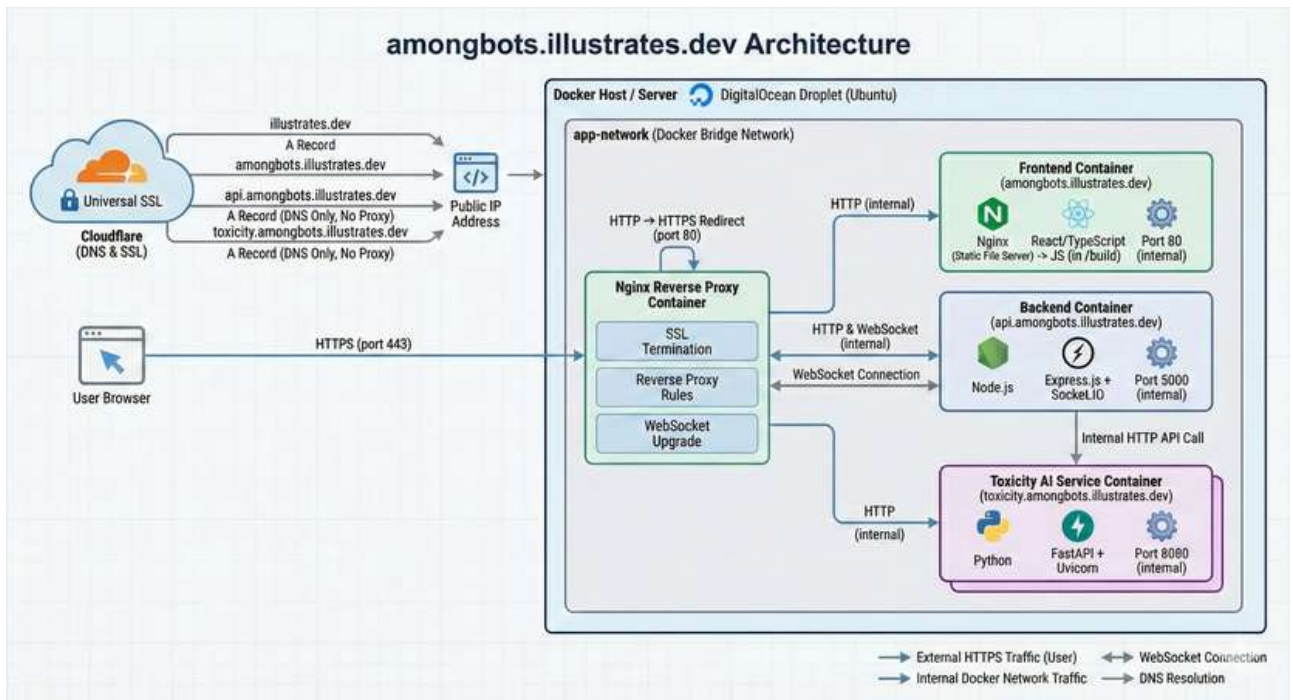
Datenschutz & Moderation

Datenschutz: Schutz der Nutzerdaten durch Anonymität; es werden keine persistenten Nutzerdaten gespeichert.

Moderation: Einsatz von Toxizitätsfiltern zur Erkennung und Eindämmung unangemessener Inhalte.

Application Architecture

Web:



Frontend:

- Erstellt mit React und TypeScript.
- Staatsverwaltung über Zustand.
- Kommuniziert mit dem Backend über Socket.IO für Echtzeit-Updates.
- Speichert minimale Sitzungsdaten (Game code, Player ID) in localStorage für die erneuten Verbindung.

Backend:

- Node.js Server mit TypeScript.
- Die Echtzeitkommunikation wird von Socket.IO übernommen.
- Spielzustand und Logik werden für jede aktive Spielsitzung im Speicher verwaltet.
- KI-Spieler sind als Teil der Spiellogik integriert und können externe APIs (z. B. OpenAI) zur Generierung von Antworten verwenden.
- Die Toxizitätsfilterung erfolgt über externe Machine-Learning-Modelle (Hugging Face-Pipelines), die über eine interne API zugänglich sind.

Datenfluss:

- Clients senden Aktionen (join, submit, vote, etc.) an den Server.

- Der Server validiert, aktualisiert den Spielstatus und sendet Änderungen an alle Clients im Spielraum.
- Kein persistenter Speicher: Alle Daten sind flüchtig und sitzungsbasiert.



Modulen

Spiel Logik

- **Rundenstart:**
 - startRoundForGame: Wählt Zielspieler, erstellt Prompt, setzt Teilnehmer.
- **Farbvergabe:**
 - Beim Beitritt oder Hinzufügen eines Spielers wird mit assignColor(game) jedem Spieler eine eindeutige Farbe zugewiesen, um Verwechslungen zu vermeiden.
- **Submission-Phase:**
 - Timer für Einreichungen, automatische Platzhalter für fehlende Beiträge.
- **Voting-Phase:**
 - Timer für Votes, KI-Votes werden automatisch geplant.
- **Scoring:**
 - Punkte für Teilnahme, Bonus für schnelle Einreichung, Malus für fehlende Beiträge.
 - Spieler mit 2+ verpassten Einreichungen werden eliminiert.
- **Elimination & Win-Conditions:**
 - Spieler mit den meisten Votes werden eliminiert (außer bei Gleichstand).
 - Das Spiel erkennt automatisch, wenn eine Seite (AIs oder Menschen) gewonnen hat, z.B. wenn alle Menschen eliminiert sind oder AIs in Überzahl sind.
- **Fast-Track für KI-Votes:**
 - Sobald alle Menschen abgestimmt haben, werden verbleibende KI-Votes sofort ausgelöst (fastTrackAIVotesForRound).
- **Farbvergabe**
 - Im Backend (gameService.ts und colorPool.ts):

Game State (Frontend)

Zustandsspeicherung mit Zustand

Im Frontend (gameStore.ts) wird der aktuelle Spielstatus mit [Zustand](#) verwaltet.

- **Wichtige Felder:**
code, playerId, alias, colorId, isHost, gameState, roundNumber, lastGameSnapshot
- **Methoden:**
 - setFromCreateOrJoin(payload): Setzt alle Felder nach Spielbeitritt/Erstellung.
 - updateFromGame(game): Aktualisiert den Status nach Server-Update.
 - reset(): Setzt den Store zurück und entfernt lokale Daten.
- **Reconnect-Logik:**
gameCode und playerId werden im localStorage gespeichert, um Reconnects nach einem Verbindungsabbruch oder Reload zu ermöglichen. Die Backend-Logik (game:reconnect in socket.ts) erlaubt es Spielern, nach einem Disconnect wieder ins Spiel einzusteigen.



Socket Events (Backend)

Socket-Events und Spielfluss

Im Backend (socket.ts) werden alle Spielaktionen über Socket.IO-Events abgewickelt:

- **game:create:** Erstellt ein neues Spiel, legt Host an, weist Farbe zu.
- **game:join:** Spieler tritt bei, erhält Farbe und ID.
- **game:addAI:** Host kann KI-Spieler hinzufügen.
- **game:start:** Host startet das Spiel, prüft Mindestanzahl an Spielern, startet erste Runde.
- **round:submit:** Spieler reichen Beiträge ein.
- **round:vote:** Spieler stimmen ab.
- **game:reconnect:** Spieler können nach Disconnect wieder beitreten.
- **game:get:** Liefert aktuellen Spielstatus.

Jede Änderung wird per emitGameUpdate an alle Clients im Raum gesendet.

- **Automatische KI-Anpassung:**
Die Funktion ensureAutoAIs passt die Anzahl der KI-Spieler dynamisch an die Zahl der menschlichen Spieler an.

AI Spieler Logik

Einbindung und Verhalten

Im Backend (aiPlayer.ts):

- **KI-Spieler werden wie menschliche Spieler behandelt,** haben aber zusätzliche Felder (isAI, aiData).
- **Automatisierte Einreichung und Voting:**

- scheduleAIForRound: Plant KI-Beiträge, sobald alle Menschen eingereicht haben.
- scheduleAIVotesForRound: Plant KI-Votes, mit Fallback-Mechanismus.
- **Toxicity-Filter:** KI sieht nur gefilterte Inhalte.
- **OpenAI-Integration:**
 - KI kann mit OpenAI-Modellen (z.B. GPT-4/5) Beiträge generieren und Voting-Entscheidungen treffen.
 - Fallback: Wenn kein Modell verfügbar, werden existierende menschliche Beiträge kopiert.
- **Team Memory für AIs:**
 KI-Spieler teilen sich Notizen und Strategien über ein gemeinsames Team-Memory (aiTeamMemory im Game-Objekt).
 Dies ermöglicht koordiniertes Verhalten und das Teilen von Informationen zwischen KI-Agenten.



Toxicity Filter

Einordnung im Gesamtprojekt

Im Projekt Among Bots übernimmt das AI Toxicity Model die Aufgabe, Chatnachrichten von Spielern und KI-Agenten automatisch zu analysieren. Ziel ist es, toxisches Verhalten zu erkennen, ohne spieltypische Aussagen wie Verdächtigungen oder Anschuldigungen fälschlich zu blockieren.

- **Sanitizing & Caching:**
 Toxicity-Bewertungen werden gecached, um Performance zu verbessern. Die Funktion sanitizeContentForAI ersetzt toxische Inhalte für KI-Spieler durch einen Platzhaltertext, basierend auf der Bewertung des Modells.

Recherche und Modellauswahl

Die Recherche erfolgte auf Hugging Face, einer Plattform für vortrainierte Machine-Learning-Modelle.

Verwendete Modelle:

- cardiffnlp/twitter-roberta-base-sentiment-latest
- MoritzLaurer/deberta-v3-large-zeroshot-v2.0

Modellfunktionen

- Das Sentiment-Modell erkennt positive, neutrale und negative Stimmungen.
- Das Zero-Shot-Modell klassifiziert Texte anhand frei definierter Labels wie toxicity, accusation oder suspicion.

Custom Labels

Für das Spiel wurden eigene Labels definiert, die speziell auf Social-Deduction-Gameplay abgestimmt sind:

```
CUSTOM_LABELS = [
```

```
    "toxicity",
```

```
    "insult",
```

```
    "harassment",
```

```
    "threat",
```

```
    "hate_speech",
```

```
    "non_toxic",
```

```
    "accusation",
```

```
    "suspicion",
```

```
]
```



Technische Umsetzung

Beide Modelle werden mit der Hugging-Face pipeline API geladen. Dadurch ist keine manuelle Tokenisierung oder Modelllogik notwendig.

Die zentrale Funktion `classify_toxicity` kombiniert beide Modelle:

```
def classify_toxicity(text):

    sentiment = sentiment_pipe(text)[0]

    zs = zs_pipeline(text, custom_labels, multi_label=True)

    return {

        "input": text,

        "sentiment_label": sentiment["label"],

        "sentiment_score": float(sentiment["score"]),

        "toxicity_scores": {

            label: float(score)
```

```
    for label, score in zip(zs["labels"], zs["scores"])\n\n}\n}
```

Mögliche Weiterentwicklungen



Erweiterung von Spielmodi und Runden: Einführung zusätzlicher Diskussionsphasen vor oder nach der Abstimmung, in denen jede Spieler:in einen eigenen Beitrag verfassen kann. Ergänzende Runden, in denen Spieler:innen gegenseitig Themen oder Schreibanweisungen vorgeben und damit sowohl andere Spieler:innen als auch die KI gezielt prompten. Weitere Rundenformate könnten zeichnerische Aufgaben, Bildbeschreibungen oder andere nicht-textuelle Interaktionen umfassen.

Finalisierung der Website: Weiterentwicklung zu einer visuell ansprechenderen Website mit Landingpage sowie verbesserten Schutzmechanismen gegen instabile oder schlechte Netzwerkverbindungen.